

Practical work 6: Vision Transformer (Vi-T) for digit sequence recognition in images

All along this classroom work we will train then test a Transformer encoder-decoder Network for the recognition of handwritten digit sequences in images. We will use new datasets for this purpose, namely:

"MNIST_5digits2D.pkl"

"MNIST_5digits2DHorizontal.pkl"

"MNIST_5digits2DVertical.pkl"

The squared images of size (120,120) pixels contain a single sequence of 5 digits organized Horizontally (class n° 10) or Vertically (class n° 11). The horizontal (resp. vertical) sequence happens at a random vertical position (resp. horizontal) in the image.

To account for the vertical or horizontal organization of the sequence two specific tokens have been introduced in the ground truth (class 10 and class 11). The Reading system will have to identify the reading order of the sequence by identifying a first reading order token as depicted in figure 1.

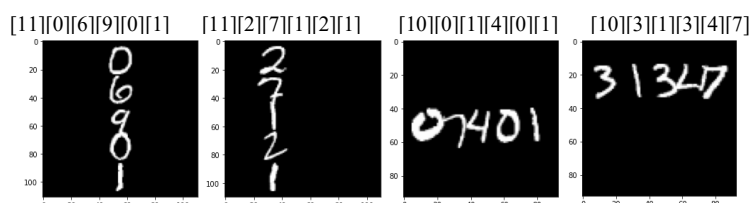


Figure 1: examples of the example from the "MNIST_5digits2D" dataset.

The MNIST_5digits2DHorizontal.pkl (resp. MNIST_5digits2DVertical.pkl) dataset contains horizontal (resp. vertical) sequences only, while the MNIST_5digits2D.pkl contains a mixture of horizontal and vertical sequences. Each dataset contains 12000 training images and 2000 testing images.

The Vision Transformer Architecture (ViT) architecture

The Encoder-Decoder architecture depicted on figure 2 below takes as input a sequence of image patches that are transformed into a sequence of feature vectors X . A 2D positional encoding is introduced to carry the original position of the path in the input image. Except the difference in pre-processing the input, the architecture remains similar to the one studied in the previous sessions, it is a standard transformer-based encoder-decoder architecture.

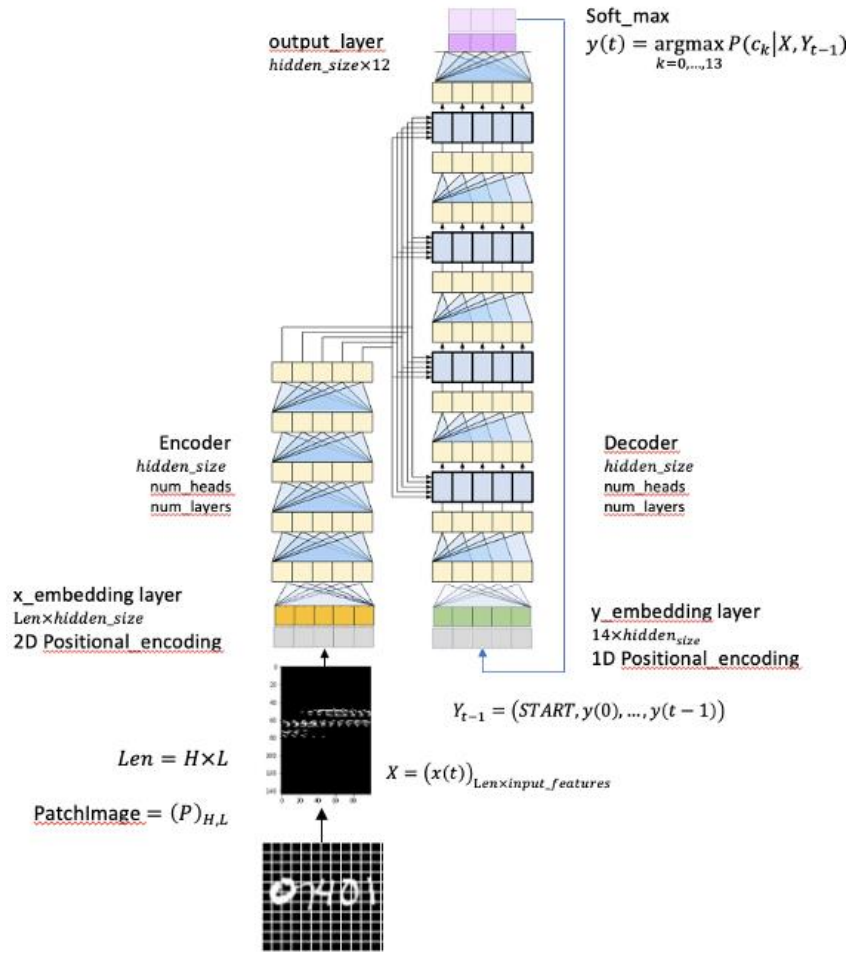


Figure 2: the ViT encoder - decoder architecture.

The 2D positional encoding is similar to the 1D positional encoding but it contains a concatenation x positional encoding and the y positional encoding. For a fixed dimension of the model ($hidden_size$), the first half dimension is used for storing the y positional encoding, while the second half stores the x positional encoding. The following equations summarize the encoding.

$$PE_{2D}(x, y, 2k) = \sin(w_k \cdot y) \quad (1)$$

$$PE_{2D}(x, y, 2k + 1) = \cos(w_k \cdot y) \quad (2)$$

$$PE_{2D}(x, y, d_{model}/2 + 2k) = \sin(w_k \cdot x) \quad (3)$$

$$PE_{2D}(x, y, d_{model}/2 + 2k + 1) = \cos(w_k \cdot x) \quad (4)$$

$$\forall k \in [0, d_{model}/4] \quad (5)$$

$$\forall k \in [0, d_{model}/4] \quad (6)$$

The image patches are obtained using a squared sliding window of size ($w_width \times w_width$). The input image of size 120 120 are serialized into a fixed length of patches of length $(120 / w_width)^2$. The patches of pixels are represented as real vectors of size $w_width \times w_width = input_features$ as represented in figure 3 below.

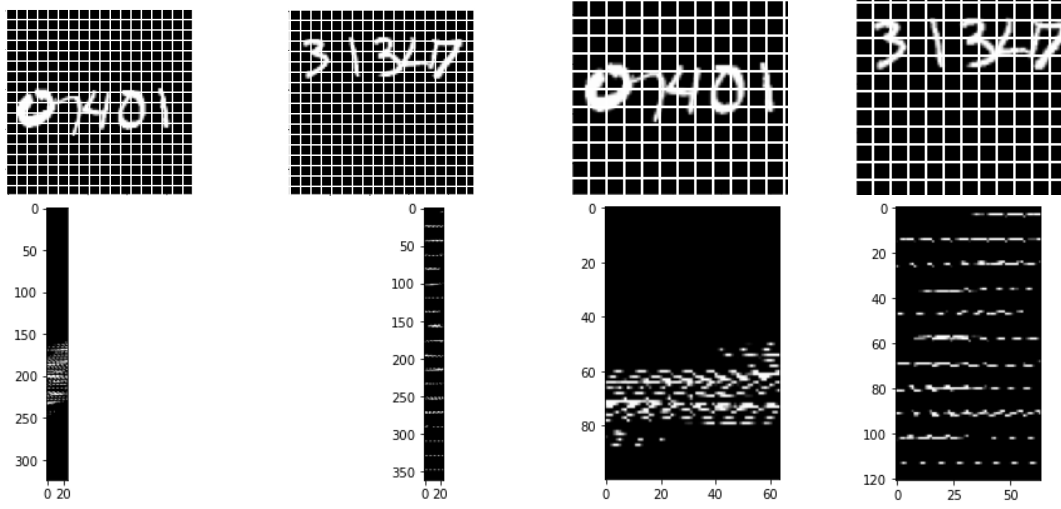


Figure 3: examples of $2D\ 5 \times 5$ sliding window (left) and 8×8 sliding window (right), with their respective sequence of embedding.

Considering the fixed dimension of the input images (120×120), the embedding sequences, as well as the embedding vectors, have a fixed size depending on the size of the sliding window as summarized in table 1 below.

Make sure you configure the code with one of these entries !

w_width	input_features	length (= max_length ²)
5	25	$576 = 24 \times 24$
8	64	$225 = 15 \times 15$
10	100	$144 = 12 \times 12$
12	144	$100 = 10 \times 10$
15	225	$64 = 8 \times 8$

Table 1: input embedding size and sequence length with respect to sliding window size.

Experimentations

- Explore the possibility to make sequence recognition for the easy (Horizontal or vertical) and for the more complex (Horizontal and vertical) dataset with a ViT architecture.

Initially, the different configurations were tested on 10,000 training examples and 1,000 validation examples. Although this amount of data is insufficient for the model to learn to generalize well in this more difficult task, it allows for faster exploration of the different configurations, since the processing time for the complete dataset is extremely long.

w width	input features	length	CRR	SRR	NB of Par.
5	25	$576 = 24 \times 24$	?	?	?
8	64	$225 = 15 \times 15$	67.45 %	0.0 %	6.356.238
10	100	$144 = 12 \times 12$	69.40 %	0.0 %	6.365.454
12	144	$100 = 10 \times 10$	70.84 %	0.0 %	6.376.718
15	225	$64 = 8 \times 8$	71.55 %	0.0 %	6.397.454

Table 2: Results for ViT with 10.000 examples of the Dataset Facile.



Figure 4: Loss over epochs for window width of 15 pixels. Other configurations presented similar distributions.

As expected for these complex models applied to a small dataset, the curves for all these training sessions showed similar behavior: unfinished training and validation suggesting overfitting.

Comparing the different sliding window sizes, we can see that the largest window obtained the best results. These larger windows produce tokens that aggregate more spatial context, facilitating the extraction of relevant visual features (e.g., the global identity of the digit). In contrast, very narrow windows excessively fragment the image, requiring the model to learn global spatial relationships only through attention, which is more difficult in regimes with little data and without pre-training.

Although smaller window widths reduce the total number of model parameters (due to smaller embeddings), they significantly increase the computational and memory cost during training.

This is because reducing the window size drastically increases the number of tokens per image, and the cost of self-attention grows quadratically with this number. As a consequence, even with fewer learnable parameters, the consumption of intermediate memory (activations, attention matrices, and gradients) exceeded the available capacity in Google Colab, leading to out-of-memory errors.

Therefore, since the larger w_width value provided the best and most stable results, it will be used for studies of the complete dataset.

Dataset	CRR	SRR
Facile	79.13 %	0.0 %
Horizontal	73.88 %	0.0 %
Vertical	72.14 %	0.0 %

Table 3: Results for ViT with 54.000 examples.



Figure 5: Loss over epochs for Horizontal dataset. Other datasets presented similar distributions.

Using all the example datasets, the curves for the three datasets also showed equivalent behavior. We see that the training could be finalized, as the validation reached convergence. Thus, we obtain a much better CRR result, but we still have errors in classifying all the strings.

This is a much more complex task than what was developed in previous TPs (because it involves translating the text, instead of a uniformly structured sequence), so there is no parameter for comparison.

We see that there is no large difference in the results for the horizontal and vertical datasets, potentially being statistically equivalent. This is expected since the ViT architecture works well with translations, as the sliding windows are arranged in a grid, without favoring a specific axis.

- Explore the capability to train a hybrid CNN-ViT architecture.

```
class CNNPatchEncoder(nn.Module):
    def __init__(self, patch_size, out_dim):
        super().__init__()

        self.patch_size = patch_size

        self.cnn = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1))
        )

        self.proj = nn.Linear(64, out_dim)

    def forward(self, x):
        B, S, D = x.shape

        x = x.view(B * S, 1, self.patch_size, self.patch_size)
        x = self.cnn(x)
        x = x.view(B * S, 64)
        x = self.proj(x)
        x = x.view(B, S, -1)

        return x
```

A simple CNN architecture with only 2 layers was defined to perform the embedding. Since cross-attention would occur during embedding, this hybrid approach has great potential to improve model performance. The CNN is efficient at capturing local strokes and digit structure, while the ViT would handle digit order and global context.

Dataset	CRR	SRR
Horizontal	75.85 %	0.0 %

Table 4: Results for CNN+ViT with 40.000 examples



Figure 5: Loss over epochs for Horizontal dataset with CNN+ViT model.

Unfortunately, even with the use of a simple CNN, the hybrid architecture resulted in considerably longer execution times. The training execution time exceeded the free limits of Google Colab before completion, impacting the evaluation across all datasets.

To try to apply the model within a feasible timeframe, only 40.000 training examples were used instead of the entire Horizontal dataset. Even so, the result obtained was superior to ViT acting alone, proving the superiority of this hybrid architecture with 54.000 examples.

Knowing that ViT depends on large amounts of data, we can say that using a smaller number of examples yielded inferior results than the entire dataset would have been capable of providing. Furthermore, it is possible that more examples would have been needed for the model to improve its SRR, which remained zero.